

..

Fundamentals of Artificial Intelligence

Prof. Dr. J. Boedecker, Prof. Dr. W. Burgard, Prof. Dr. F. Hutter, Prof. Dr. B. Nebel, Dr. rer. nat. M. Tangermann
M. Krawez, T. Schulte Sommer
Semester 2019

University of Freiburg
Institute of Computer Science

..

Exercise Sheet 1 — Solutions

Task 1.1 (Possibilities and Limitations of AI)

Research in the AI literature or on the Internet to find out to what extent the following problems can be solved today using computers or robots:

- (a) Playing the board games checkers and go.

Solution:

Checkers is completely solved, optimal game guarantees a tie.

The game of Go has been regarded as a benchmark for AI for a long time. Since the number of possible games is much larger than, eg, in chess, it was often claimed that the game requires human-like creativity. In 2016, Google's AlphaGo algorithm has beaten Lee Sedol, a world top Go player.
http://en.wikipedia.org/wiki/Computer_Go.

..

- (b) Real-time natural language processing.

Solution:

Natural language processing is still an open field of research. A major milestone recently was IBM's Watson computer, which was able to win the first place in the popular quiz show Jeopardy! after competing against former human winners. These involved understanding questions posed in natural language and searching for the answer through a large database of information.

<http://www.research.ibm.com/deepqa/deepqa.shtml>

- (c) Autonomy of unmanned vehicles and aircraft (UGVs and UAVs).

Solution:

In the context of the "DARPA Grand Challenge" in 2005 the team of Sebastian Thrun from Stanford University reached the goal of auto-nomous navigation of a vehicle through the desert at a route of 200 km.

By 2016, Google's Waymo cars have driven autonomously for over 2.7 million kilometers on public roads, with only a few accidents caused by the self-driving car.

More information:

http://en.wikipedia.org/wiki/DARPA_Grand_Challenge and http://en.wikipedia.org/wiki/Google_driverless_car.

Autonomous aircraft are comparably easy to construct:

<http://www.ida.liu.se/ext/witas/> or http://en.wikipedia.org/wiki/Unmanned_aerial_vehicle.

(d) Automatic facial recognition.

Solution:

In a crowd the rate of correctly recognized people is around 80 percent.

In well-controlled settings this rate can be high enough to be used for, eg, access control in banks, in military or scientific facilities. In these use cases the person has to face the camera frontally and don't move for some time.

(e) Playing computer games (e.g. classic Atari games) like a human.

Solution:

The performance is of course dependent on the game at hand, but a large number of games can be approached by deep learning:

<http://www.nature.com/nature/journal/v518/n7540/full/nature14236.html>

(f) Composing music.

Solution:

Several approaches to automatic music generation exist, two most prominent algorithms are:

EMI

In a training phase the algorithm analyzes several music pieces, then it tries to produce a new piece following the style of the input pieces. <http://www.computerhistory.org/atcm/algorithmic-music-david-cope-and-emi>

Melomics

Instead of copying a given style, Melomics generates a new music piece from some seed value, following general rules of music composition.

<http://geb.uma.es/melomics>

(g) Turing test

Solution:

Chat bots (programs which simulate a conversation by answering to user input) have a long history and have grown quite sophisticated in recent years. They typically use a large online data base, which is extended by analyzing chats with users. However, those bots are still far away from passing the Turing test, mainly because they cannot well understand the semantics and context of a conversation.

Write down your findings in 2-3 sentences each.

Task 1.2 (Performance and Benefits)

- (a) What is the difference between a performance measurement and a utility function?

Solution:

A performance measurement evaluates the performance of an agent from the outside, from the perspective of an objective external instance, while a utility function allows the agent itself to evaluate its possible actions.

- (b) Describe the relationship between performance measurement and Utility function for a learning agent.

Solution:

A learning agent changes its utility function due to the jerk report of the "critic", which in turn depends on the performance measurement.

Task 1.3 (Rational Agents)

- (a) For each of the following agents, give a description of the performance dimensions, the environment, the actuators and the sensors (performance Environment Actuators Sensors) to:

- (i) Playing Monopoly
- (ii) Athlete doing long jump
- (iii) Play 2048 (<http://gabrielecirulli.github.io/2048>)

Solution:

agent	P	E	A	S
Monopoly	possession end of the game	after game board, other players	dice, " properties buy, ...	dice eyes, " actions other players, ...
long jump	cracked width in meters	sports stadium	muscles of the athletes	Eyes des athletes, referee's decision (transgress, " width)
2048	highest tile, high score	4 x 4 grid	4 movements state of	game

- (b) Classify the agent environments formalized in (a) according to following criteria:

- fully observable or partially observable

- deterministic or stochastic
- static or dynamic
- discrete or continuous

Solution:

- Monopoly: partially observable, stochastic, static, discrete • Long jump: fully observable, stochastic, static (apart from the wind), continuous
- 2048: fully observable, stochastic, static, discrete

Task 1.4 (Problem Formalization)

For each of the following problems, provide a formulation that is as precise as possible, consisting of an initial state, state space, actions, goal test and a path cost function:

- You want to solve the Rubik's Cube. <http://de.wikipedia.org/wiki/Zauberw%C3%BCrfel>

Solution:

States: The states are $9 \cdot 6 = 54$ -tuples of the color of the faces.

Initial state: "random" configuration of colors

Target test: Do all sides have the same color?

Actions: Rotate one of the side surfaces by 90° (top, bottom, left, right, front, back) possibly also rotates by 180° and the middle surface

Path cost: Depending on metric and available actions, 1 per action (or 2 per 180° rotation)

- You want to color a map of Europe using only four colors. In order to be able to recognize the borders of individual countries, it is necessary that there are no neighboring countries with the same color.

Solution:

States: The states are 49-tuples of the colors red, green, blue, yellow or uncolored (eg). Each entry represents a country on the map.

For the sake of compactness, the colors can be coded with the numbers 0 (red) to 4 (uncolored).

Initial state: All countries are uncolored / all countries are red / each country can have one of the $4+1=5$ colors [uncolored, r,g,b,y].

Target test: All countries are colored, neighboring countries have different colors?

Actions: Assign one of the 4 colors to a country and check beforehand whether a neighboring country already has the same color / recolor a country if a conflict is detected.

Path Costs: Each action incurs a unit cost.

Grundlagen der Künstlichen Intelligenz

Prof. Dr. J. Boedecker, Prof. Dr. W. Burgard, Prof. Dr. F. Hutter, Prof. Dr. B. Nebel,
Dr. rer. nat. M. Tangermann
M. Krawez, T. Schulte
Sommersemester 2019

Universität Freiburg
Institut für Informatik

Übungsblatt 1 — Lösungen

Aufgabe 1.1 (Möglichkeiten und Grenzen der KI)

Recherchieren Sie in der KI-Literatur bzw. im Internet, inwiefern folgende Probleme heutzutage mittels Computer- bzw. Robotereinsatz gelöst werden können:

- (a) Spielen der Brettspiele Dame und Go.

Lösung:

Checkers is completely solved, optimal game guarantees a tie.

The game of Go has been regarded as a benchmark for AI for a long time. Since the number of possible games is much larger than, e.g., in chess, it often was claimed that the game requires human-like creativity. In 2016, Google's AlphaGo algorithm has beaten Lee Sedol, a world top Go player.

http://en.wikipedia.org/wiki/Computer_Go.

- (b) Verarbeiten natürlicher Sprache in Echtzeit.

Lösung:

Natural language processing is still an open field of research. A major milestone recently was IBM's Watson computer, which was able to win the first place in the popular quiz show Jeopardy! after competing against former human winners. This involved understanding questions posed in natural language and searching for the answer through a large database of information.

<http://www.research.ibm.com/deepqa/deepqa.shtml>

- (c) Autonomie unbemannter Fahr- und Flugzeuge (UGVs und UAVs).

Lösung:

In the context of the „DARPA Grand Challenge“ in 2005 the team of Sebastian Thrun from the Stanford University reached the goal of autonomous navigation of a vehicle through the desert at a route of 200 km.

By 2016, Google's Waymo cars have driven autonomously for over 2.7 millions kilometers on public roads, with only few accidents caused by the self-driving car.

More information:

http://en.wikipedia.org/wiki/DARPA_Grand_Challenge and
http://en.wikipedia.org/wiki/Google_driverless_car.

Autonomous aircrafts are comparably easy to construct:

<http://www.ida.liu.se/ext/witas/> or
http://en.wikipedia.org/wiki/Unmanned_aerial_vehicle.

- (d) Automatische Gesichtserkennung.

Lösung:

In a crowd the rate of correctly recognized people is around 80 percent. In well-controlled settings this rate can be high enough to be used for, e.g., access control in banks, in military or scientific facilities. In these use cases the person have to face the camera frontally and don't move for some time.

- (e) Spielen von Computerspielen (z.B. klassische Atari-Spiele) wie ein Mensch.

Lösung:

The performance is of course dependent on the game at hand, but a big number of games can be approached by deep learning:

<http://www.nature.com/nature/journal/v518/n7540/full/nature14236.html>

- (f) Komponieren von Musik.

Lösung:

Several approaches to automatic music generation exist, two most prominent algorithms are:

EMI

In a training phase the algorithm analyses several music pieces, then it tries to produce a new piece following the style of the input pieces.

<http://www.computerhistory.org/atc/m/algorithmic-music-david-cope-and-emi>

Melomics

Instead of copying a given style, Melomics generates a new music piece from some seed value, following general rules of music composition.

<http://geb.uma.es/melomics>

- (g) Turing-Test

Lösung:

Chat bots (programs which simulate a conversation by answering to user input) have a long history and have grown quite sophisticated in recent years. They typically use a large online data base, which is extended by analyzing chats with users. However, those bots are still far away from passing the Turing test, mainly because they cannot well understand the semantics and context of a conversation.

Schreiben Sie Ihre Erkenntnisse in jeweils 2–3 Sätzen auf.

Aufgabe 1.2 (Performanz und Nutzen)

- (a) Was ist der Unterschied zwischen einer Performanzmessung und einer Nutzenfunktion?

Lösung:

Ein Performanzmessung bewertet die Leistung eines Agenten von außen, quasi aus der Sicht einer objektiven externen Instanz, während eine Nutzenfunktion dem Agenten selbst erlaubt, seine möglichen Aktionen zu bewerten.

- (b) Beschreiben Sie den Zusammenhang zwischen Performanzmessung und Nutzenfunktion bei einem lernenden Agenten.

Lösung:

Ein lernender Agent verändert seine utility function auf Grund der Rückmeldung des „Kritikers“, die wiederum von der Performanzmessung abhängt.

Aufgabe 1.3 (Rationale Agenten)

- (a) Geben Sie für jeden der folgenden Agenten eine Beschreibung des Performanzmaßes, der Umgebung, der Aktuatoren und der Sensoren (**P**erformance **E**nvironment **A**ctuators **S**ensors) an:

- (i) Monopoly spielen
- (ii) Leichtathlet beim Weitsprung
- (iii) 2048 spielen (<http://gabrielecirulli.github.io/2048>)

Lösung:

Agent	P	E	A	S
Monopoly	Besitz nach Spielende	Spielbrett, andere Spieler	Würfeln, Grundstücke kaufen, ...	Würfelaugen, Handlungen anderer Spieler, ...
Weitsprung	gesprungene Weite in Meter	Sportstadion	Muskeln des Athleten	Augen des Athleten, Schiedsrichterentscheidung (übertreten, Weite)
2048	höchstes Highscore	Tile, 4 x 4 grid	4 Bewegungen	Zustand des Spiels

- (b) Klassifizieren Sie die in (a) formalisierten Umgebungen der Agenten nach folgenden Kriterien:

- vollständig beobachtbar oder teilweise beobachtbar

- deterministisch oder stochastisch
- statisch oder dynamisch
- diskret oder kontinuierlich

Lösung:

- Monopoly: teilweise beobachtbar, stochastisch, statisch, diskret
- Weitsprung: vollständig beobachtbar, stochastisch, statisch (abgesehen vom Wind), kontinuierlich
- 2048: vollständig beobachtbar, stochastisch, statisch, diskret

Aufgabe 1.4 (Problemformalisierung)

Geben Sie für die folgenden Problemstellungen jeweils eine möglichst präzise Formulierung an, die aus Anfangszustand, Zustandsraum, Aktionen, Zieltest und einer Pfadkostenfunktion besteht:

- Sie wollen den Rubiks Zauberwürfel lösen.

<http://de.wikipedia.org/wiki/Zauberw%C3%BCrfel>

Lösung:

Zustände: Die Zustände sind $9 \cdot 6 = 54$ -Tupel der Farbe der Seitenflächen.

Anfangszustand: “zufällige” Konfiguration der Farben

Zieltest: Haben alle Seitenflächen eine einheitliche Farbe?

Aktionen: Drehen einer der Seitenflächen um 90° (Oben, unten, links, rechts, vorne, hinten) eventuell auch Drehungen um 180° sowie der Mittelfläche

Pfadkosten: Je nach Metrik und verfügbaren Aktionen 1 pro Aktion (oder 2 pro 180° Drehung)

- Sie wollen eine Karte von Europa mit nur vier Farben einfärben. Damit man die Grenzen einzelner Länder erkennen kann, ist es erforderlich, dass es keine Nachbarländer mit gleicher Einfärbung gibt.

Lösung:

Zustände: Die Zustände sind 49-Tupel der Farben *rot*, *grün*, *blau*, *gelb* oder *ungefärbt* (z.B.). Jeder Eintrag repräsentiert ein Land auf der Karte. Der Kompaktheit halber kann man die Farben mit den Zahlen 0 (*rot*) bis 4 (*ungefärbt*) kodieren.

Anfangszustand: Alle Länder sind ungefärbt / alle Länder sind rot / jedes Land kann eine der $4+1=5$ Farben [ungefärbt, r,g,b,y] haben.

Zieltest: Alle Länder sind gefärbt, benachbarte Länder haben unterschiedliche Farben?

Aktionen: Einem Land eine der 4 Farben zuweisen und vorher überprüfen ob ein Nachbarland schon die gleiche Farbe hat / ein Land umfärben, wenn ein Konflikt festgestellt wird.

Pfadkosten: Jede Aktion verursacht Einheitskosten.

Fundamentals of Artificial Intelligence

Prof. Dr. J. Boedecker, Prof. Dr. W. Burgard, Prof. Dr. F. Hutter, Prof. Dr. B. fog,

Dr. rer. nat. M. Tangermann

M. Krawez, T. Schulte

summer semester 2019

University of Freiburg
Institute of Computer Science

Exercise Sheet 2 — Solutions

Task 2.1 (Informed Search)

A household robot wants to determine the shortest path from S (start) to G (destination). The robot can move one horizontal or vertically connected cell, provided that it is not separated by a wall (thick black line) from the current cell. Every movement has uniform cost of 1. Figure (a) shows the initial state, Figure (b) the heuristic values.

	a	b	c	d	e
5					
4					
3		S			
2			G		
1					

(a) Initial state

	a	b	c	d	e
5	5	4	3	4	5
4	4	3	2	3	4
3	3	2	1	2	3
2	2	1	0	1	2
1	3	2	1	2	3

(b) Heuristic values

- (a) Determine A. Enter^y-Search to find the shortest path from S to G the f-values of all generated nodes into the corresponding cells in Figure (a). All other cells should be free remain.

Solution:

	a	b	c	d	e
5	8	6	6	8	
4	6	4	6	8	
3	4	S	6	8	
2	4	2	G		
1	6	4	4	6	8

(e1 is optional)

- (b) Give the definition of a feasible heuristic. Is the heuristic from figure (b) permissible?

Solution:

A heuristic h is admissible if $h(n) \leq h^*(n)$ for all n , where h^* is the optimal heuristic. I.e. h is admissible if it never overestimates the cost of the cheapest solution from n to a goal. The heuristic shown in figure (b) is admissible. (It's the Manhattan Distance heuristic.)

- (c) How many nodes must A^* expand if h is used as a heuristic where $h(n)$ is the actual cost of an optimal plan of n to the destination G ?

Solution:

Depending on the implementation, either 6 or 7 (depending on whether the target state is expanded or not). The implementation from the lecture requires 7 expansions (b3, b4, b5, c5, c4, c3, c2).

Task 2.2 (Local Search)

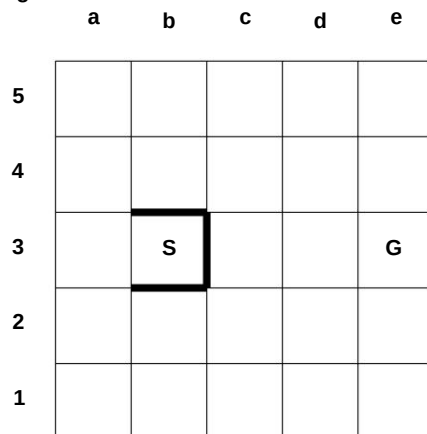
We will now discuss hill climbing in the same context (robot navigation on a grid with walls as obstacles).

- Explain how hill-climbing would be performed as a method of reaching a certain point on the plane.
- Explain with the help of an example why non-convex obstacles (consisting of several walls) to a local maximum for the algorithm can lead.
- Is it possible to arrive at a non-optimal solution for convex obstacles? to get there?
- Was simulated annealing always performed for this family of problems from local Find out the maxima? Explain your answer.

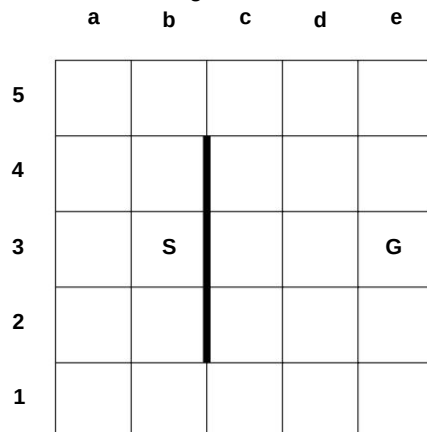
Solution:

(a) Hill-climbing is surprisingly effective at finding reasonable if not optimal paths for very little computational cost, and seldom fails in two dimensions. Let $d(x, y)$ be the straight-line distance between point x and point y , and let the value of a point p be $\hat{y}d(p, \text{goal})$. An agent can move to any vertex accessible by a single straight-line movement, and when hill-climbing it will choose the accessible vertex nearest the goal.

(b) This can occur when an agent is at a non-convex obstacle as shown in the figure below.



(c) It is possible but unlikely. This requires that the agent be at a vertex that is closer to the goal than any of the other vertices of the obstacle, as illustrated in the figure below.



(d) If a solution exists (the agent can't be entirely walled-in) and one chooses an appropriate annealing schedule, simulated annealing can escape locally maximum for this family of problems.

Task 2.3 (Search algorithms)

Prove the following statements:

(a) Breadth-first search is a special case of uniform cost search.

Solution:

Breadth-first search always expands an unexpanded node of minimal depth, while uniform cost search always expands an unexpanded node with minimal path costs. If the action costs in uniform cost search are constant 1, then a node has depth k if and only if its path costs are k . In particular, it has minimal depth if and only if it has minimal path costs. The expansion criteria are therefore identical.

(b) Breadth-first search, depth-first search and uniform cost search are special cases of greedy best-first search.

Solution:

It has already been shown in the last subtask that breadth-first search is a special case of uniform cost search. It remains to show that depth-first search and uniform cost search are special cases of greedy best search.

Depth-first search is a special case of greedy best search for $h(n) := \text{depth}(n)$.
Uniform cost search is a special case of greedy best search for $h(n) := g(n)$

(c) Uniform cost search is a special case of A* search.

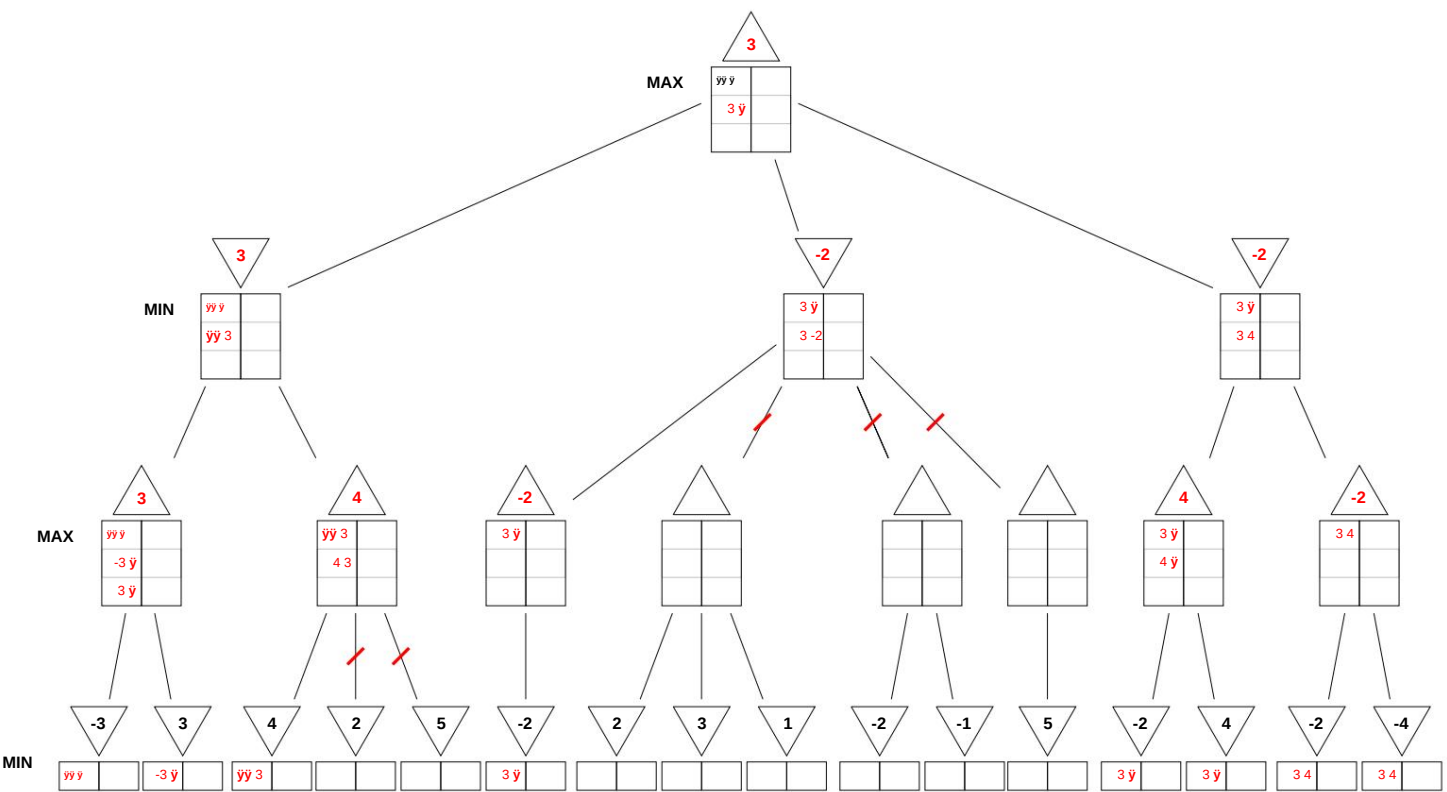
Solution:

The evaluation function for uniform cost search is $f(n) = g(n)$, for A* search $f(n) = g(n) + h(n)$. So set $h \leq 0$.

Task 2.4 (Board games)

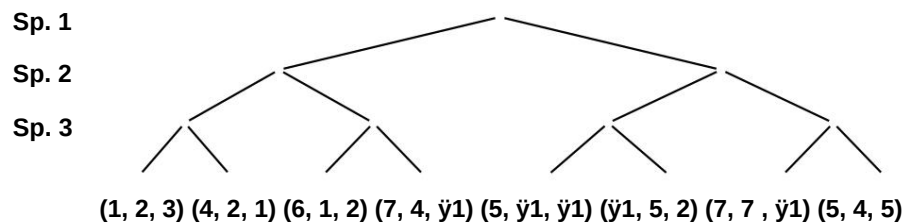
(a) Consider the following game tree for a two-player game.

Simulate the behavior of the minimax algorithm with β - γ pruning (expanding child nodes from left to right). Enter the values of the calculated nodes into the triangles and the β - γ intermediate values into the corresponding tables.



- (b) Now consider the problem of evaluating the game tree of a three-person game that does not necessarily satisfy the zero-sum condition. You may assume that no alliances between players are allowed. The players are called 1, 2 and 3. In contrast to two-person zero-sum games, the evaluation function now returns triples (x_1, x_2, x_3) , where x_i is the value for player i .

Complete the game tree by annotating all interior nodes and the root node with the corresponding value triples.



Solution:

- Level Sp 3: 123, 612, -152, 545 •
- Level Sp 2: 123, -152 •
- Level Sp 1: 123

..

- (c) Suppose that the value triple (5, 4, 5) on the far right has been replaced by (5, 4, y_1). What difficulty now arises in evaluating the game tree? Suggest how the evaluation of a node can be modified, given the evaluations of its successors, so that at the root node a

"robust" result.

Solution:

..

Problem: Decision for player 3 is no longer clear.

Pessimistic / Optimistic / Sets

Task 2.5 (Forward Checking / Edge Consistency)

Consider the 6-queens problem, where 6 pieces are to be placed on a 6×6 board in such a way that no two queens are on the same horizontal, vertical or diagonal line. The range of values is $\text{dom}(v_i) = 1, \dots, 6$ for all variables $v_i \in V$. Now consider the state $\tilde{y} = \{v_1 \tilde{y} 2, v_2 \tilde{y} 5\}$.

	v_1	v_2	v_3	v_4	v_5	v_6
1						
2	Q					
3						
4						
5	Q					
6						

- (a) Create edge consistency in γ . In particular, specify the value ranges of the variables before and after creating the edge consistency. You can assume that the value range of variables with already assigned values consists only of the assigned value, while unassigned variables have the full range of values. Select always the variable with the lowest index for which no edge consistency has yet been generated.
- ..
- (b) Perform forward checking in γ . Compare the result with (a).

Solution:

(a)

The table shows the domains after generating the edge consistency for all constraints in which the respective variable is involved.

	v1	v2	v3	v4	v5	v6
AC-v1 = AC-v2 {2}	{5}	{123456}	{123456}	{123456}	{123456}	
AC-v3		{2}	{5}	{13}	{123456}	{123456}
AC-v4 {2} {5} {13}		{146}	{123456}	{123456}		
AC-v5 {2} {5} {13}		{146}	{4}	{123456}		
AC-v4 {2} {5} {13}		{16}	{4}	{123456}		
AC-v6 {2} {5} {13}		{16}	{4}	{6}		
AC-v3 {2} {5} {1}		{16}	{4}	{6}		
AC-v4 {2} {5} {1}		{4}	{6}			

Whenever the domain of a variable is reduced, all constraints in which the variable is involved are put back into the queue. This can then lead to further changes.

..

(In the second run, AC- v_i no longer stands for all of the constraints of v_i , only for those that have been reinserted into the queue)

(b)

	v1	v2	v3	v4	v5	v6
FC dom(v) {2} {5} {13} {146} {134} {346}						

If you check the arc consistency of v_5 , you will find that v_3 no longer has any allowed values when you assign $v_5 \hat{=} 1$ or $v_5 \hat{=} 3$. This is not detected by forward checking. Arc-Consistency also has a queue and propagates changes to a domain so that the constraints of a variable may be tested multiple times. In this case, ..

AC-3 immediately shows that the constraints are unsatisfiable, while the search algorithm has much more to do with forward checking.

Grundlagen der Künstlichen Intelligenz

Prof. Dr. J. Boedecker, Prof. Dr. W. Burgard, Prof. Dr. F. Hutter, Prof. Dr. B. Nebel,
Dr. rer. nat. M. Tangermann
M. Krawez, T. Schulte
Sommersemester 2019

Universität Freiburg
Institut für Informatik

Übungsblatt 2 — Lösungen

Aufgabe 2.1 (Informierte Suche)

Ein Haushalts-Roboter will den kürzesten Pfad von S (Start) nach G (Ziel) bestimmen. Der Roboter kann sich in jedem Schritt um eine horizontal oder vertikal verbundenen Zelle fortbewegen, sofern diese nicht durch eine Wand (dicke schwarze Linie) von der aktuellen Zelle getrennt ist. Jede Bewegung hat uniforme Kosten von 1. Abbildung (a) zeigt den Initialzustand, Abbildung (b) die Heuristikwerte.

	a	b	c	d	e
5					
4					
3		S			
2			G		
1					

(a) Initialzustand

	a	b	c	d	e
5	5	4	3	4	5
4	4	3	2	3	4
3	3	2	1	2	3
2	2	1	0	1	2
1	3	2	1	2	3

(b) Heuristikwerte

- (a) Führen Sie eine A*-Suche durch, um den kürzesten Pfad von S nach G zu bestimmen. Tragen Sie von allen generierten Knoten die f -Werte in die entsprechenden Zellen in Abbildung (a) ein. Alle anderen Zellen sollen frei bleiben.

Lösung:

	a	b	c	d	e
5	8	6	6	8	
4	6	4	6	8	
3	4	S	6	8	
2	4	2	G		
1	6	4	4	6	8

(e1 is optional)

- (b) Geben Sie die Definition einer *zulässigen Heuristik* an. Ist die Heuristik aus Abbildung (b) zulässig?

Lösung:

A heuristic h is admissible if $h(n) \leq h^*(n)$ for all n , where h^* is the optimal heuristic. I.e. h is admissible if it never over estimates the cost of the cheapest solution from n to a goal. The heuristic shown in figure (b) is admissible. (It's the Manhattan Distance heuristic.)

- (c) Wieviele Knoten muss A* expandieren, wenn h^* als Heuristik verwendet wird, wobei $h^*(n)$ die tatsächlichen Kosten eines optimalen Plans von n zum Ziel G angibt?

Lösung:

Je nach Implementation entweder 6 oder 7 (je nachdem ob der Zielzustand expandiert wird oder nicht). Die Implementierung aus der Vorlesung benötigt 7 Expansionen (b3, b4, b5, c5, c4, c3, c2).

Aufgabe 2.2 (Lokale Suche)

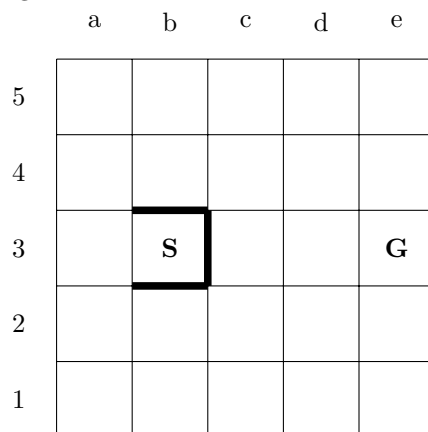
Wir werden nun *Hill-Climbing* im selben Zusammenhang (Roboternavigation auf einem Grid mit Wänden als Hindernisse) untersuchen.

- Erklären Sie, wie *hill-climbing* als Methode zum Erreichen eines bestimmten Punktes in der Ebene durchgeführt werden würde.
- Erläutern Sie mit Hilfe eines Beispiels, warum nicht-konvexe Hindernisse (bestehend aus mehreren Wänden) zu einem lokalen Maximum für den Algorithmus führen können.
- Ist es möglich, bei konvexen Hindernissen zu einer nicht-optimalen Lösung zu gelangen?
- Würde *simulated annealing* bei dieser Problemfamilie immer aus lokalen Maxima herausfinden? Begründen Sie Ihre Antwort.

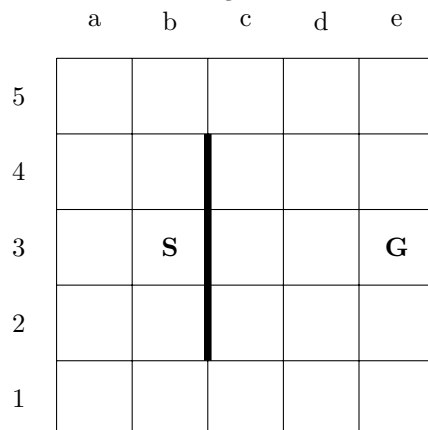
Lösung:

- (a) Hill-climbing is surprisingly effective at finding reasonable if not optimal paths for very little computational cost, and seldom fails in two dimensions. Let $d(x, y)$ be the straight-line distance between point x and point y , and let the value of a point p be $-d(p, goal)$. An agent can move to any vertex accessible by a single straight-line movement, and when hill-climbing it will choose the accessible vertex nearest the goal.

- (b) This can occur when an agent is at a non-convex obstacle as shown in the figure below.



- (c) It is possible but unlikely. This requires that the agent be at a vertex that is closer to the goal than any of the other vertices of the obstacle, as illustrated in the figure below.



- (d) If a solution exists (the agent can't be entirely walled-in) and one chooses an appropriate annealing schedule, simulated annealing can escape local maxima for this family of problems.

Aufgabe 2.3 (Suchalgorithmen)

Beweisen Sie die folgenden Aussagen:

- (a) Breitensuche ist ein Spezialfall der uniformen Kostensuche.

Lösung:

Breitensuche expandiert immer einen unexpandierten Knoten minimaler Tiefe, während uniforme Kostensuche immer einen unexpandierten Knoten mit minimalen Pfadkosten expandiert. Sind die Aktionskosten bei uniformer Kostensuche konstant 1, so hat ein Knoten genau dann Tiefe k , wenn seine Pfadkosten k betragen. Insbesondere hat er also minimale Tiefe genau dann, wenn er minimale Pfadkosten hat. Die Expansionskriterien sind somit identisch.

- (b) Breitensuche, Tiefensuche und uniforme Kostensuche sind Spezialfälle der gierigen Bestensuche (greedy best-first search).

Lösung:

Dass Breitensuche ein Spezialfall der uniformen Kostensuche ist, wurde schon in der letzten Teilaufgabe gezeigt. Es bleibt also noch zu zeigen, dass Tiefensuche und uniforme Kostensuche Spezialfälle der gierigen Bestensuche sind.

Tiefensuche ist ein Spezialfall der gierigen Bestensuche für $h(n) := -\text{depth}(n)$.

Uniforme Kostensuche ist ein Spezialfall der gierigen Bestensuche für $h(n) := g(n)$

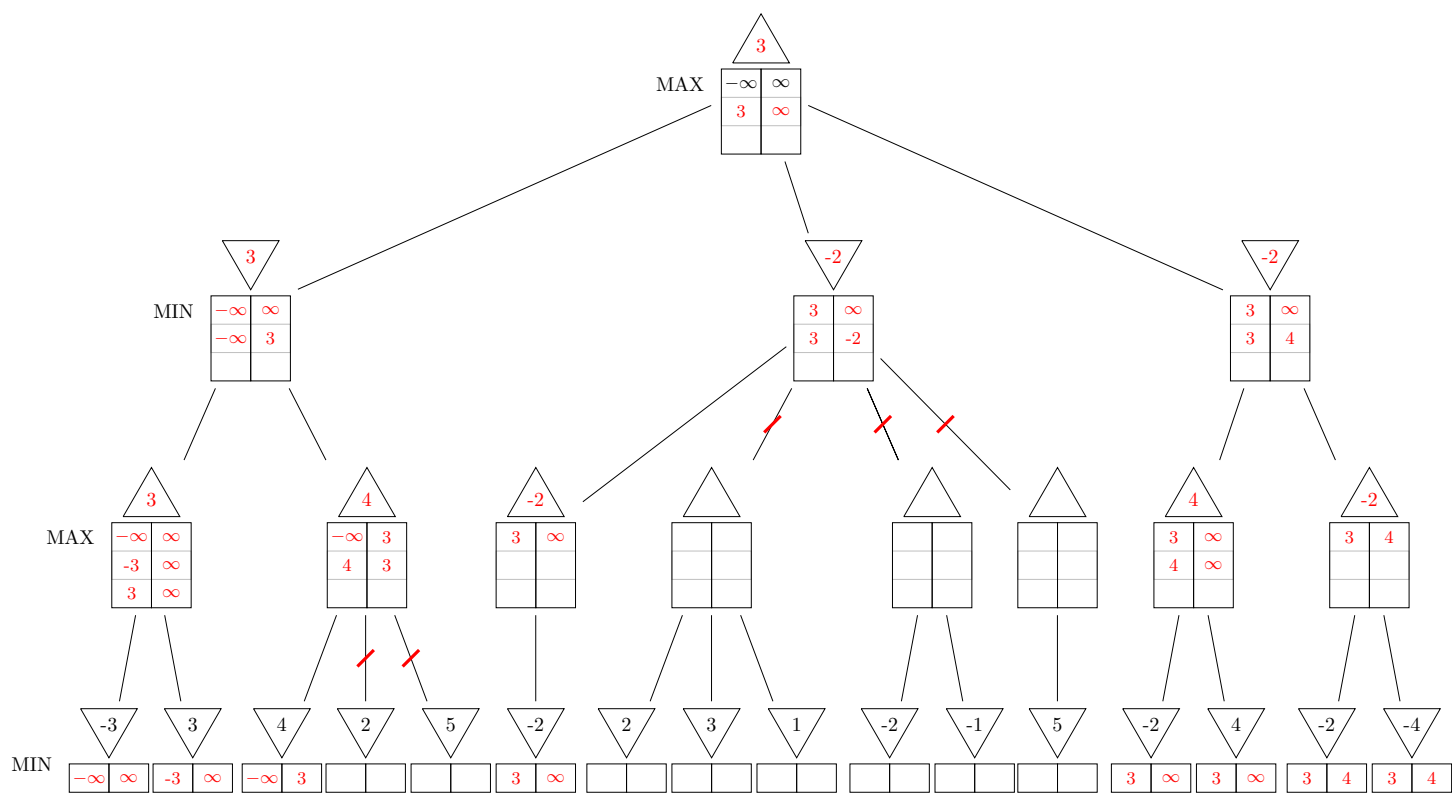
- (c) Uniforme Kostensuche ist ein Spezialfall der A*-Suche.

Lösung:

Die Bewertungsfunktion bei uniformer Kostensuche ist $f(n) = g(n)$, bei A*-Suche $f(n) = g(n) + h(n)$. Setze also $h \equiv 0$.

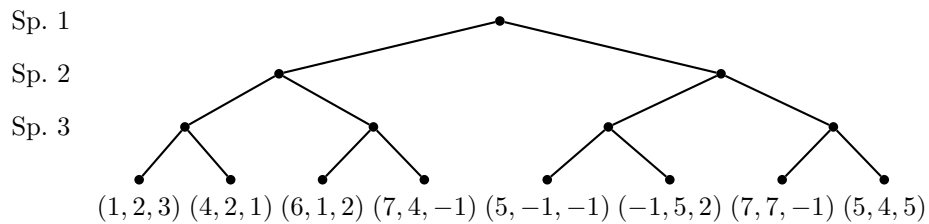
Aufgabe 2.4 (Brettspiele)

- (a) Betrachten Sie folgenden Spielbaum für ein Zwei-Personen-Spiel. Simulieren Sie das Verhalten des Minimax Algorithmus mit α - β -Pruning (expandieren Sie Kindknoten dabei von links nach rechts). Tragen Sie die Werte der berechneten Knoten in die Dreiecke und die α - β -Zwischenwerte in die zugehörigen Tabellen ein.



- (b) Betrachten Sie nun das Problem, den Spielbaum eines Drei-Personen-Spiels zu evaluieren, das nicht notwendigerweise die Nullsummenbedingung erfüllt. Sie dürfen annehmen, dass keine Allianzen zwischen Spielern erlaubt sind. Die Spieler heißen 1, 2 und 3. Im Gegensatz zu Zwei-Personen-Nullsummenspielen liefert die Bewertungsfunktion nun Tripel (x_1, x_2, x_3) zurück, wobei x_i der Wert für Spieler i ist.

Vervollständigen Sie den Spielbaum, indem Sie alle inneren Knoten und den Wurzelknoten mit den entsprechenden Wert-Tripeln annotieren.



Lösung:

- Ebene Sp 3: 123, 612, -152, 545
- Ebene Sp 2: 123, -152
- Ebene Sp 1: 123

- (c) Angenommen, das Wert-Tripel $(5, 4, 5)$ ganz rechts würde durch $(5, 4, -1)$ ersetzt. Welche Schwierigkeit tritt nun bei der Auswertung des Spielbaums auf? Schlagen Sie vor, wie die Auswertung eines Knotens gegeben die Auswertungen seiner Nachfolger modifiziert werden kann, damit man am Wurzelknoten ein „robustes“ Ergebnis erhält.

Lösung:

Problem: Entscheidung für Spieler 3 nicht mehr eindeutig.

Pessimistisch / Optimistisch / Sets

Aufgabe 2.5 (Forward Checking / Kantenkonsistenz)

Betrachten Sie das 6-Damen Problem, bei dem 6 Spielfiguren auf einem 6×6 Felder großen Brett so platziert werden sollen, dass sich keine zwei Damen auf der selben horizontalen, vertikalen oder diagonalen Linie befinden. Der Wertebereich sei $\text{dom}(v_i) = 1, \dots, 6$ für alle Variablen $v_i \in V$. Betrachten Sie nun den Zustand $\alpha = \{v_1 \mapsto 2, v_2 \mapsto 5\}$.

	v_1	v_2	v_3	v_4	v_5	v_6
1						
2	♔					
3						
4						
5		♔				
6						

- (a) Erzeugen Sie Kantenkonsistenz in α . Geben Sie hierzu insbesondere die Wertebereiche der Variablen vor und nach dem Erzeugen der Kantenkonsistenz an. Sie dürfen annehmen, dass der Wertebereich von Variablen mit bereits zugewiesenen Werten nur aus dem zugewiesenen Wert besteht, während unbelegte Variablen den vollen Wertebereich haben. Wählen Sie immer diejenige Variable mit niedrigstem Index, für welche noch keine Kantenkonsistenz erzeugt wurde.
- (b) Führen Sie Forward-Checking in α aus. Vergleichen Sie das Ergebnis mit (a).

Lösung:

(a)

In der Tabelle sind die Domänen nach dem Erzeugen der Kantenkonsistenz für alle Constraints an denen die jeweilige Variable beteiligt ist.

	v_1	v_2	v_3	v_4	v_5	v_6
AC- $v_1 = ACv_2$	{2}	{5}	{123456}	{123456}	{123456}	{123456}
AC- v_3	{2}	{5}	{13}	{123456}	{123456}	{123456}
AC- v_4	{2}	{5}	{13}	{146}	{123456}	{123456}
AC- v_5	{2}	{5}	{13}	{146}	{4}	{123456}
AC- v_4	{2}	{5}	{13}	{16}	{4}	{123456}
AC- v_6	{2}	{5}	{13}	{16}	{4}	{6}
AC- v_3	{2}	{5}	{1}	{16}	{4}	{6}
AC- v_4	{2}	{5}	{1}	{}	{4}	{6}

Immer wenn die Domäne einer Variablen reduziert wird werden alle Constraints an denen die Variable beteiligt ist wieder in die *queue* gesteckt. Dies kann dann zu weiteren Änderungen führen.

(Im zweiten Durchlauf steht AC- v_i nicht mehr für alle Constraints von v_i , sondern nur für die, die wieder in die *queue* eingefügt wurden)

(b)

	v_1	v_2	v_3	v_4	v_5	v_6
FC $dom(v)$	{2}	{5}	{13}	{146}	{134}	{346}

Wenn man die Arc Consistency von v_5 checkt, stellt man fest, dass v_3 keine erlaubten Werte mehr hat, wenn man $v_5 \mapsto 1$ oder $v_5 \mapsto 3$ zuweist. Dies wird durch forward checking nicht erkannt. Arc-Consistency hat zudem eine *queue* und propagiert Änderungen an einer Domäne weiter so dass die Constraints einer Variable eventuell mehrfach getestet werden. In diesem Fall findet AC-3 sofort heraus, dass die Constraints unerfüllbar sind, während der Such-Algorithmus mit Forward-Checking noch deutlich mehr zu tun hat.